

STATISTICAL METHODS FOR MACHINE LEARNING

Experimental Project

Done by: Chaima Ben Fredj

ID: 56374A

Assignment 5

Clustering and Initialization

K-Means vs. K-Means++

University of Milan

Data Science for Economics

A.Y. 2025/2026

Declaration:

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1	Introduction	2
2	Methodology	2
2.1	Standard K-Means	2
2.2	K-Means++ Initialization	2
2.3	Implementation Details	3
3	Datasets	3
3.1	Synthetic Dataset	3
3.2	Digits Dataset	3
4	Empirical Evaluation	4
5	Results	4
5.1	Visualization of Initialization	4
5.2	Best and Worst Case Outcomes	4
5.3	Inertia Distribution over 50 Trials	5
5.4	Variability Comparison	6
5.5	Effect of the Number of Clusters	6
6	Interpretation of Results	7
6.1	Synthetic Dataset: Initialization Matters	7
6.2	Digits Dataset: High Dimensions Change the Picture	7
6.3	Stability vs Performance	7
7	Conclusion	7

1 Introduction

Clustering is as a fundamental unsupervised learning task, focusing on dividing a dataset into groups of comparable points. The K-Means algorithm [1] is among the most popular clustering methods because it is both simple and efficient. This method involves a two-step process: first, it allocates each data point to the closest centroid, and then it recalculates each centroid to represent the average of the points assigned to it. The goal is to minimize the *inertia*, which is calculated as the sum of squared distances from each point to its assigned centroid [2]:

$$\mathcal{L}(C) = \sum_{j=1}^k \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2 \quad (1)$$

where C_j is the j -th cluster and $\boldsymbol{\mu}_j$ is its centroid.

A widely recognized limitation of K-Means is its sensitivity to initialization. Since the objective is non-convex, the algorithm can converge to a poor local minimum depending on the initial placement of centroids. Running the algorithm twice on the same dataset with different random seeds can result in substantially different inertia values.

In this project we want to study the impact of initialization on clustering performance by comparing standard K-Means (uniform random initialization) with K-Means++ [1], which uses a carefully designed probabilistic seeding strategy. We implement both algorithms from scratch and evaluate them across 50 independent trials on two datasets: a synthetic dataset with well-separated clusters, and the Digits dataset, a real-world high-dimensional dataset of handwritten digit images.

2 Methodology

2.1 Standard K-Means

The standard K-Means algorithm proceeds as follows:

1. **Initialize:** Select k centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k$ by sampling k points uniformly at random from the dataset.
2. **Assign:** For each point $\mathbf{x}_i$, assign it to the nearest centroid:

$$c_i = \arg \min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

3. **Update:** Recompute each centroid as the mean of its assigned points:

$$\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$$

4. **Repeat** steps 2–3 until convergence [4] (i.e., the centroids shift by less than a tolerance $\epsilon = 10^{-4}$).

2.2 K-Means++ Initialization

K-Means++ [1] replaces only the initialization step with a smarter seeding procedure, while the rest of the algorithm (assign and update steps) remains identical to the standard K-Means. The initialization proceeds as follows:

1. Choose the first centroid μ_1 uniformly at random from the dataset.
2. For each remaining point $\mathbf{x}$, compute its distance to the nearest already-chosen centroid:

$$D(\mathbf{x}) = \min_{j \in \text{chosen}} \|\mathbf{x} - \mu_j\|$$

3. Sample the next centroid with probability proportional to $D(\mathbf{x})^2$:

$$\Pr[\text{choose } \mathbf{x}] = \frac{D(\mathbf{x})^2}{\sum_{\mathbf{x}'} D(\mathbf{x}')^2}$$

4. Repeat steps 2–3 until k centroids have been chosen.

The $D(\mathbf{x})^2$ weighting is the key insight: points far from existing centroids are more likely to be selected as the next centroid, naturally spreading the initial centroids across the dataset. Arthur and Vassilvitskii prove that this initialization yields an expected cost within $O(\log k)$ of the optimal clustering, regardless of the data [1].

2.3 Implementation Details

We implemented both algorithms from scratch in Python using only `numpy` package for numerical computation. The implementation uses:

- Vectorized distance computation via broadcasting: `np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)`
- Empty cluster management: if a cluster loses all its points during an update step, the centroid is kept at its previous position.
- Convergence criterion: the algorithm will stop when the total centroid shift $\|\mu^{(t+1)} - \mu^{(t)}\| < 10^{-4}$, or after a maximum of 300 iterations to prevent the algorithm from running forever.

3 Datasets

3.1 Synthetic Dataset

We generated a synthetic dataset by using `sklearn.datasets.make_blobs` with $n = 1500$ samples, $k = 5$ clusters, and standard deviation $\sigma = 0.8$ per cluster. This dataset is two-dimensional, allowing us direct visualization of clustering results. The well-separated clusters make it an excellent environment to see the impact of initialization: when random initialization places two centroids in the same cluster, the resulting partition is clearly not optimal and easily detectable.

3.2 Digits Dataset

The Digits dataset (`sklearn.datasets.load_digits`) is composed of 1797 images of handwritten digits (0–9). Each image is 8×8 pixels, resulting in a feature vector with 64 dimensions. The pixel values were normalized to $[0, 1]$ by dividing by 16. We perform clustering with $k = 10$, which corresponds to the number of digit categories.

This dataset serves as our high-dimensional evaluation: the 64 features make it impossible to visualize directly, and the class structure is less clearly separated than the synthetic dataset. It tests whether the advantage of K-Means++ persists for real-world data.

4 Empirical Evaluation

In order to obtain a reliable statistical comparison, we run both algorithms with 50 different random trials (0 through 49). Each trial produces a different initialization, leading to a different final clustering. We record the final inertia for each trial and compare the distributions between the algorithms.

We will calculate the following metrics:

- **Mean inertia:** average quality in all trials.
- **Standard deviation:** for measuring variability of outcomes: a lower value means that the algorithm is more stable and predictable.
- **Worst/Best ratio:** ratio of the worst inertia to the best. A ratio close to 1.0 means a good performance regardless of initialization.

5 Results

5.1 Visualization of Initialization

Figure 1 illustrates how the two initialization strategies place starting centroids on the synthetic dataset. Random initialization (panel b) can cluster two stars in nearby regions, leaving other parts of the data uncovered. K-Means++ initialization (panel c) distributes the five starting centroids across the full extent of the data, with each star landing in a distinct region.

Figure 1 — Initialization Strategies on Synthetic Data

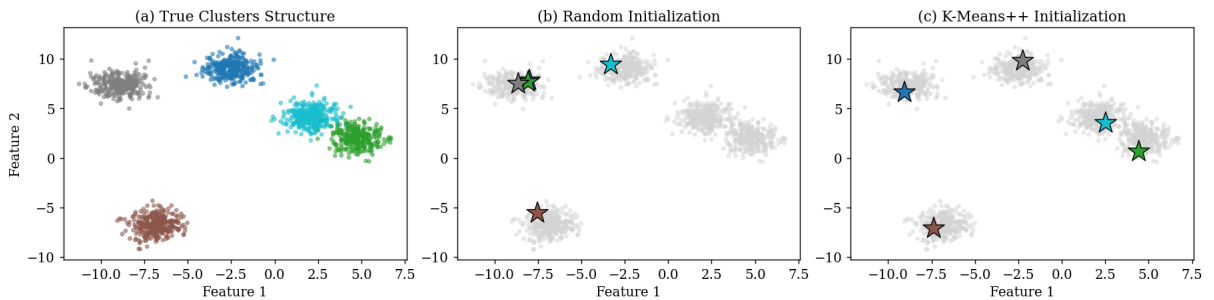


Figure 1: Initialization strategies on the synthetic dataset. (a) True Clusters Structure. (b) Random initialization: centroids may cluster together. (c) K-Means++ initialization: centroids spread across the dataset due to $D(\mathbf{x})^2$ weighting.

5.2 Best and Worst Case Outcomes

Figure 2 shows the best and worst clustering outcomes for each algorithm across 50 trials on the synthetic dataset. The worst K-Means run (*top right*) clearly merges two distinct clusters, producing an unbalanced partition with poor inertia. In contrast, the worst K-Means++ run (*bottom right*) is visually very close to the best run, confirming that the smart initialization avoids catastrophic failures.

Figure 2 — Best and Worst Clustering Results (50 trials)

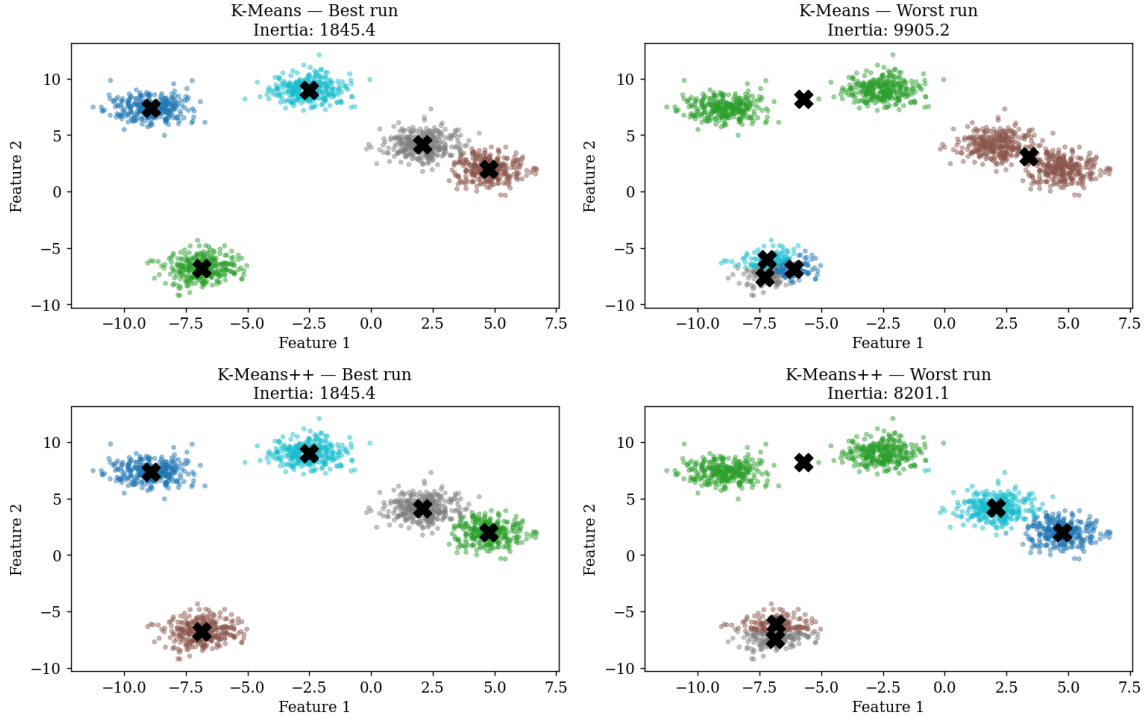


Figure 2: Best and worst clustering results over 50 trials on the sythetic dataset. K-Means worst-case outcomes are significantly worse than those of K-Means++.

5.3 Inertia Distribution over 50 Trials

Figure 3 shows the distribution of final inertia values across 50 trials for both datasets.

Figure 3 — Distribution of Inertia over 50 Trials

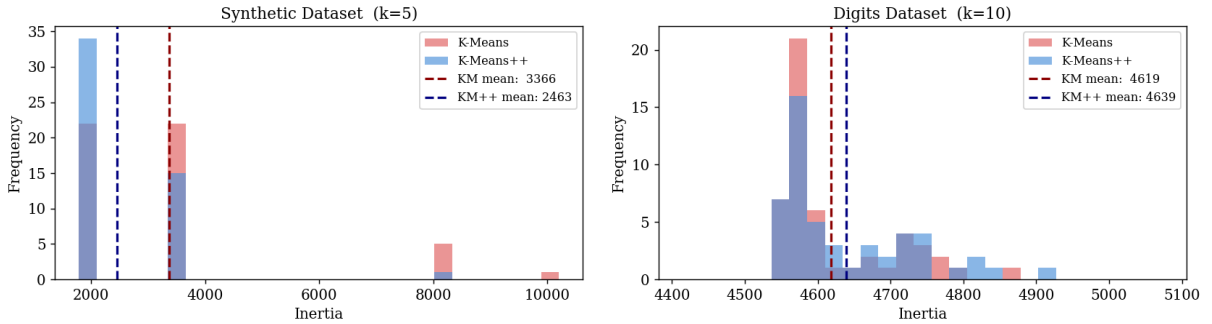


Figure 3: Distribution of inertia over 50 trials. Left: synthetic dataset. Right: Digits dataset. Dashed lines indicate the mean for each algorithm.

On the synthetic dataset (left panel), K-Means produces a wide spread of inertia values with a heavy right tail, reflecting the frequent occurrence of bad initializations. K-Means++ is concentrated around lower inertia values with significantly less spread. On the Digits dataset (right panel), both distributions overlap considerably, with no meaningful difference between the two algorithms.

5.4 Variability Comparison

Figure 4 presents boxplots summarizing the variability of both algorithms. Table 1 shows the summary statistics of the numerical results.

Figure 4 — Inertia Variability over 50 Trials

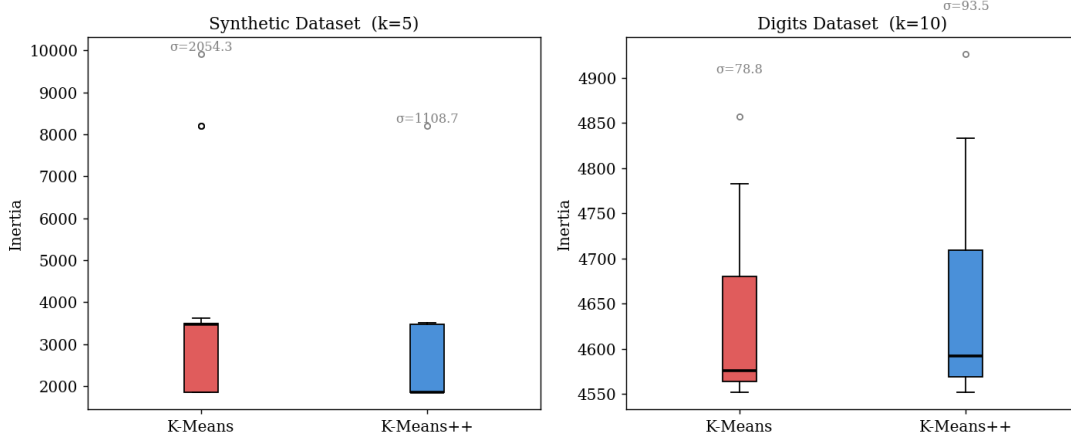


Figure 4: Inertia variability over 50 trials. The σ annotation shows the standard deviation of each distribution.

Table 1: Summary statistics over 50 independent trials.

Dataset	Algorithm	Mean	Std Dev	Min	Max
Synthetic ($k = 5$)	K-Means	3365.89	2054.31	1845.44	9905.18
	K-Means++	2463.39	1108.66	1845.44	8201.11
Digits ($k = 10$)	K-Means	4618.56	78.81	4551.34	4856.80
	K-Means++	4638.87	93.54	4551.36	4925.85

5.5 Effect of the Number of Clusters

Figure 5 shows how the mean inertia and its variability evolve as k increases on the synthetic dataset. The shaded bands represent ± 1 standard deviation across 50 trials.

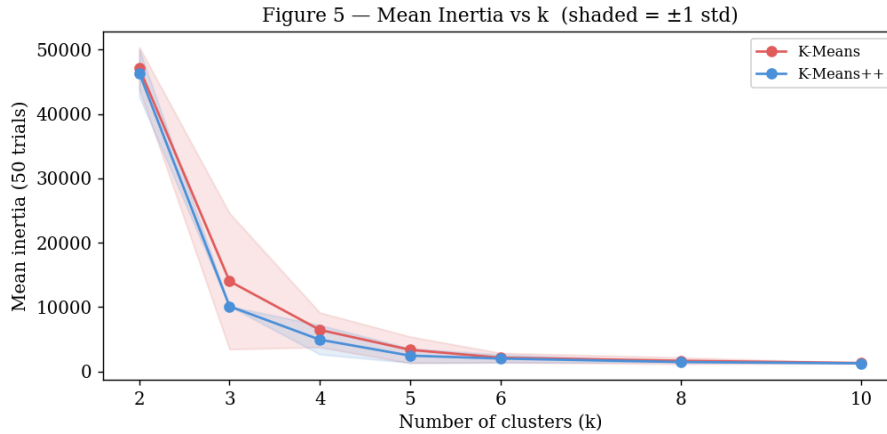


Figure 5: Mean inertia vs number of clusters k on the synthetic dataset. Shaded bands show ± 1 standard deviation over 50 trials.

6 Interpretation of Results

6.1 Synthetic Dataset: Initialization Matters

On the synthetic dataset, K-Means++ achieves a **26.8% lower mean inertia** and **almost half the standard deviation** compared to standard K-Means (2054 vs 1108). Since both algorithms use the same assign and update steps, we can interpret that the improvement is mainly the result of a better initialization: K-Means++ rarely places two starting centroids in the same true cluster.

This result is consistent with the theoretical guarantee from Arthur and Vassilvitskii [1], who proved that $D(\mathbf{x})^2$ seeding systematically avoids poor initializations — something random sampling cannot guarantee. Our experiments reflect this: **K-Means++** not only achieves lower mean inertia, but it actually does so consistently across the trials, which is again consistent with what the theory predicts.

6.2 Digits Dataset: High Dimensions Change the Picture

On the Digits dataset, K-Means++ performed **marginally worse** than standard K-Means: its **mean inertia was slightly higher** (4638.87 vs 4618.56, a difference of 0.4%) and its **standard deviation was also larger** (93.54 vs 78.81). However, these differences are too small to be practically meaningful. This is an interesting negative result. In a high dimensions dataset, the true clusters (digit classes) are not as well-separated as the clusters of the synthetic dataset. Any reasonable initialization tends to be similarly distant from the true cluster centers, so both random and smart initialization methods tend to have similar results.

This finding actually aligns with the known phenomenon that K-Means clustering becomes harder in high dimensions [5].

6.3 Stability vs Performance

An important observation is that the *best-case* inertia of both algorithms is nearly identical on both datasets (1845.44 vs 1845.44 for synthetic). The advantage of K-Means++ lies not in finding a better solution, but in finding it *reliably* — even the worst K-Means++ run tends to be close to the best run, while standard K-Means can fail badly. In practice, this means K-Means++ is preferable whenever only a small number of restarts are affordable.

7 Conclusion

To conclude, this project aimed to investigate a simple but important question: does the way we initialize K-Means actually matter? The short answer is yes, *but only under the right conditions*. We implemented K-Means and K-Means++ from scratch and conducted a comparison between 50 trials on two datasets. The main findings are:

1. On the synthetic dataset, where clusters are well-separated and the correct partitioning is unambiguous, K-Means++ made a clear difference, it found better clusterings and provides lower mean inertia and much lower variance. The smart initialization essentially keeps the algorithm away from the bad starting points that random seeding can fall into.
2. On the Digits dataset, however, the picture changed. Both algorithms performed almost identically (with K-Means showing marginally lower mean inertia, a difference too small to be meaningful.) — not because K-Means++ stopped working, but because the data itself is harder. In 64 dimensions, the digit classes overlap and blend into each other, so there is no single "correct" clustering waiting to be found; Suggesting that the benefit of smart initialization diminishes when cluster boundaries are less distinct.

Perhaps the most interesting finding is that K-Means++ does not necessarily find a *better* solution than standard K-Means because their best case inertia is nearly identical. What it does is make good solutions *easier to reach*, reducing the risk of wasting a run on a poor local minimum. In practice, this means that fewer restarts are needed to get a reliable result.

These results confirm that initialization is a critical factor in the reliability of K-Means, and that K-Means++ is a simple and effective remedy on structured, low-dimensional data.

References

- [1] D. Arthur and S. Vassilvitskii. *k-means++: The Advantages of Careful Seeding*. In Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 1027–1035, 2007.
- [2] Rykov, A. and De Amorim, R. and Makarenkov, V. and Mirkin, B. *Inertia-Based Indices to Determine the Number of Clusters in K-Means: An Experimental Evaluation*. IEEE Access, pages 11761–11773, 2024.
- [3] S. P. Lloyd. *Least Squares Quantization in PCM*. IEEE Transactions on Information Theory, 28(2):129–137, 1982.
- [4] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [5] R. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961.